

Laboratorio di Calcolo

A.A. 2026/2027, Canale A-Dec

Giacomo Artoni, Lorenzo Caprini

Linguaggi di alto livello

I livelli di comunicazione

Un processore comprende unicamente stati elettrici (0 e 1). Per rendere possibile la programmazione, si utilizzano strati successivi che "nascondono" i dettagli fisici:

- Livello Fisico/Hardware: Porte logiche, transistor.
- Linguaggio Macchina: Istruzioni binarie native della CPU.
- Assembly: Simboli testuali che mappano 1:1 il codice macchina.
- Alto Livello: Astrazioni logiche e matematiche (C, Python).

Linguaggio macchina ed Assembly a confronto

- Linguaggio Macchina (Binario/Esadecimale):
  - Costituito da byte (es. B8 2A 00 00 00).
  - È l'unico formato che la CPU esegue fisicamente.
  - Impraticabile per lo sviluppo umano.
- Linguaggio Assembly:
  - Assegna un nome "mnemonico" (facile da ricordare) a ogni istruzione numerica (`mov` , `add`).
  - Usa identificatori leggibili per i registri hardware (es. `eax`).
  - Non è portabile: ogni famiglia di processori (Intel, ARM) ha il proprio Assembly.

Aspetto	**Macchina**	**Assembly**
Sintassi	Numerica	Testuale
Traduttore	Nessuno	Assembler
Esempio	89 E5	`mov rbp, rsp`
Esecuzione	Immediata	Dopo assemblaggio

Dal basso all'alto livello

Limiti dei linguaggi di basso livello

- Zero Portabilità: un programma scritto per un chip specifico non gira su un altro senza essere riscritto da capo.
- Micro-gestione: Serve specificare manualmente ogni spostamento di memoria, registro e indirizzo.
- Tempi di sviluppo: scrivere algoritmi complessi richiede migliaia di righe di codice estremamente verboso.

Vantaggi nel passaggio a linguaggi di alto livello

- Lo stesso file sorgente può essere convertito per Mac, Windows o Linux.
- Uso di variabili, cicli (for, while) e funzioni al posto di registri e salti hardware.
- Sintassi simile alla matematica e all'inglese, molto più facile da correggere e aggiornare.

Come si può lavorare con codice di alto livello?

Introduzione al C: Scrittura e Primo Programma

I File Sorgente in C

- **Testo puro:** I programmi C si scrivono con un qualsiasi editor di testo (`VS Code` , `emacs` , `gedit` , `nano`).
- **Estensioni convenzionali:**
  - `.c` : Contiene l'implementazione e la logica del codice sorgente.
  - `.h` : File di intestazione (*header*) per dichiarazioni di funzioni e librerie.
- **Il Compilatore:** Traduce il sorgente in eseguibile. Nel corso useremo `gcc` (*GNU C Compiler*).

Il Primo Sorgente (`hello.c`)

```c
#include <stdio.h>

int main() {
    // Sommiamo degli interi,
    // stampandone il risultato
    int a = 2;
    int b = 3;
    int sum = a + b;

    printf("a + b = %d\n", sum);

    return 0;
}
```

Workflow di compilazione

Invocazione di `gcc` indicando il sorgente e il nome dell'eseguibile

```
$ gcc -o hello hello.c
```

- `gcc` : Invoca il compilatore
- `-o hello` : Imposta il nome dell'eseguibile generato
- `hello.c` : File di codice sorgente

```
$ ls -l     # Controlla la creazione dell'eseguibile
-rwxr-xr-x  1 giacomo  staff  33432 Oct  5 14:33 hello
-rw-r--r--  1 giacomo  staff    197 Oct  5 14:33 hello.c
```

```
$ ./hello   # Esegue il programma
a + b = 5
```

Analisi degli Errori

In caso di errori di sintassi gcc indica file, riga e colonna:

```
$ gcc -o hello hello.c
hello.c: In function ‘main’:
hello.c:8:5: error: expected ‘,’ or ‘;’ before ‘printf’
   8 |     printf("a + b = %d\n", sum);
     |     ^~~~~~
```

Consiglio:

Correggere sempre gli errori partendo dal *primo* riportato in alto dall’output.

Analisi del Primo Programma C

- `#include` : direttiva del preprocessore per caricare le funzioni di I/O standard (`printf`).
- `int main()` : Punto d'ingresso dell'esecuzione del programma.
- `// ...` : Commento su una singola riga (ignorato dal compilatore).
- `int a = 2;` : Dichiarazione e inizializzazione di una variabile intera.
- `int sum = a + b;` : Dichiarazione ed assegnazione con espressione aritmetica.
- `printf(...)` : Chiamata di funzione con specificatore di formato `%d` .
- `return 0;` : Codice ri ritorno per dire al sistema operativo che il prorgamma è stato eseguito con successo (0) oppure se qualche errore è stato riscontrato (!=0).

```c
#include <stdio.h>

int main() {
    // Sommiamo degli interi,
    // stampandone il risultato
    int a = 2;
    int b = 3;
    int sum = a + b;

    printf("a + b = %d\n", sum);

    return 0;
}
```